Høgskulen på Vestlandet

DAT107 - Databaser

Video 13: NoSQL: Graf Databaser

Introduksjon til NoSQL

Dag Larsen
Oppdatert av Lasse Jenssen

Læringsmål

Kjenne til de **vanligste egenskapene til grafbaserte NoSQL-databaser**, og gi eksempler på bruksområder.

Kjenne de mest karakteristiske egenskapene ved den grafbaserte databasen **Neo4J.**

Litteratur

NoSQL

Læreboka:

- ✓ Kapittel 15.3 - NoSQL Landskapet
  - 15.3.3 – NoSQL-familiene
- ✓ Kapittel 15.5 - Grafdatabasen Neo4J
  - 15.5.1 – Graf = Noder + Kanter
  - 15.5.2 – Problemet med «grafspørringer» i relasjonsdatabaser
  - 15.5.3 – Enkle grafspørringer med Cypher
  - 15.5.4 – WHERE betingelser
  - 15.5.5 – Innsetting, oppdatering og sletting
  - 15.5.6 – Komplekse mønstre, aggregering og grafalgoritmer

Høgskulen på Vestlandet

Typer av NoSQL databaser

En liten oversikt

Høgskulen på Vestlandet

Type		Value Eksempel	Løsninger
Nøkkel-Verdi (key-value pair)	Nøkkel/verdi-par	"Elev:1:Fornavn": "Bill"	**Amazon DynamoDB**, **Redis**, Riak
Kolonne	Nøkkel/verdi-par, men ulike kolonner eller kolonnefamilier blir lagret hver for seg	(Kommer senere)	**Cassandra**, Hbase
Dokument	Et dokument blir representert som et BSON-objekt (Binary JSON) og organiseres i dokumentsamlinger.	{ VareNr: "10830", Produkt: "Nisseskjegg", Pris : 66.50, Antall: 42 }	**MongoDB**, ElasticSearch, CouchDB
Graf/ node	En graf (et «nettverk») er bygd opp fra noder og kanter.	(Kommer senere)	**Neo4J**
Vektor	Laget for å lagre og søke i **høydimensjonale vektorer**, typisk brukt i maskinlæring, AI og semantiske søk	Utenfor pensum	Milvu sourc (manag

Høgskulen på Vestlandet

Graf databaser

Typer av NoSQL

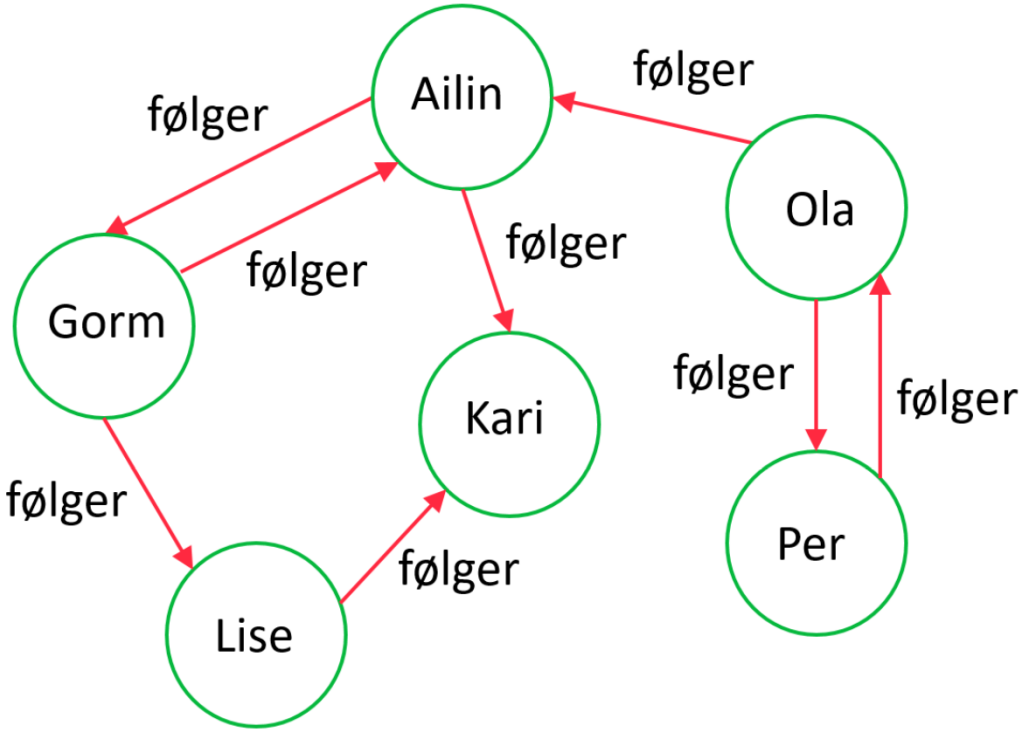
Graf databaser

Typer av NoSQL databaser

Høgskulen på Vestlandet

- ✓ En graf (et «nettverk») er bygd opp fra **noder** og **kanter**.
- ✓ Slike databaser egner seg for domener som er naturlig graf-aktige, f.eks. **sosiale nettverk** – hvilke personer «følger» hvilke andre på Twitter.

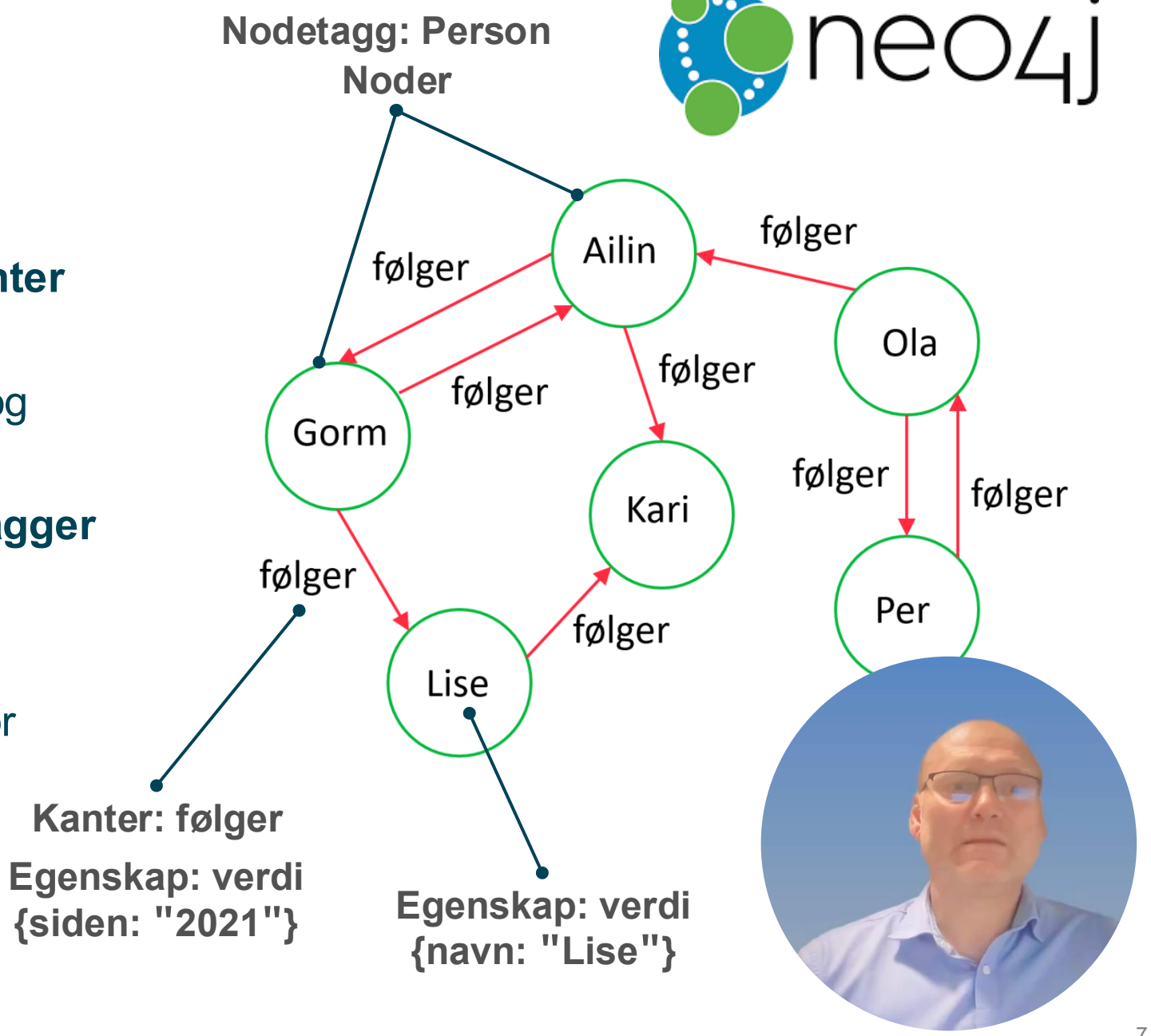
Eksempler:
Neo4J, Microsoft Azure Cosmos DB.



Neo4J

Graf database

- ✓ Neo4J er en grafdatabase (av **noder** (ovalene) og rettede **kanter** (linjene).
- ✓ Noder og kanter har **egenskaper** og hver egenskap en **verdi**.
- ✓ Noder kan gis en eller flere **nodetagger** (eller «merkelapper»).
- ✓ Bruker **Cypher** Query Language for spørringer.



Neo4J & Cypher

Graf database

- ✓ **ASCII-kunst** er en måte å lage «bilder» ved å sette sammen symbolene i tegnsettet ASCII. Eksempel: :-)
- ✓ **Cypher** :
  - For å hente ut data fra grafdatabaser brukes en slags ASCII-kunst for å beskrive mønstre.
  - Mønster som beskriver to noder sammenkoblet med en kant: **( )--->( )**
  - Spørring som returnerer alle person par, der den ene følger den andre – spørreresultatet er også en graf:

```
MATCH (p1:Person)-[f:FØLGER]->(p2:Person)
   RETURN p1, f, p2;
```

Cypher: CREATE

CRUD Operasjoner

- ✓ Opprett en node

```
CREATE (p:Person:Foreleser {fornavn: "Lasse", etternavn: "Jenssen"});
CREATE (p:Emne {kode: ”DAT107", navn: ”Databaser"});
```

- ✓ Opprett flere noder

```
CREATE (p1:Person:Foreleser {fornavn: "Dag", etternavn: "Toppe Larsen"}),
       (p2:Person:Foreleser {fornavn: "Lars-Petter", etternavn: "Helland"});
```

- ✓ Opprette relasoner

```
MATCH (f:Foreleser {fornavn: "Lasse"}), (e:Emne {kode: ”DAT107"})
CREATE (f)-[:UNDERVISER]->(e);
```

Cypher: READ

CRUD Operasjoner

- ✓ Finn noder med relasjoner

```
MATCH (n1)-[r]->(n2) RETURN n1, r, n2;
```

- ✓ Finn alle forelesere

```
MATCH (p:Foreleser) RETURN p;
```

- ✓ Finn alle emner som “Dag” underviser

```
MATCH (p:Person {fornavn: ”Dag"})-[r:UNDERVISER]->(e:Emne)
      RETURN p, r, e;
```

- ✓ Finn alle roller som “Lars-Petter” har I henold til de ulike emnene

```
MATCH (p:Person {fornavn: "Lars-Petter"})-[r]-> (e)
      RETURN p, r, e;
```

Cypher: UPDATE & DELETE

CRUD Operasjoner

- ✓ Oppdater (etternavnet til person med fornavn «Lasse»):

```
MATCH (p:Person {fornavn:"Lasse"}) SET p.etternavn = "Jenssen";
```

- ✓ Slett (foreleser med fornavn «Lasse»):

```
MATCH (f:Foreleser {fornavn:"Lasse"}) DELETE f;
```

ORDER BY statement, Alias & Tabellform

Neo4J & Cypher

- ✓ **ORDER BY** fungere tilsvarende som i SQL

- ✓ Ved å hente ut egenskaper (Etternavn) i RETURN blir spørreresultatet på **tabellform**.

```
MATCH (f:Foreleser)-[r:UNDERVISER]->(e:Emne)
RETURN f.etternavn AS Etternavn, e.kode AS kode, e.navn AS Navn
ORDER BY f.etternavn;
```

```
+-------------------------------------------------------------+
| Etternavn      | kode     | Navn                            |
+-------------------------------------------------------------+
| "Helland"      | "DAT100" | "Grunnleggende programmering" |
| "Helland"      | "DAT107" | "Databaser"                     |
| "Jenssen"      | "DAT107" | "Databaser"                     |
| "Toppe Larsen" | "DAT107" | "Databaser"                     |
+-------------------------------------------------------------+
```

Hente ut data som Graf

Neo4J & Cypher

- ✓ Vi kan også hente ut data i **Graf-form**

```
MATCH p=(f:Foreleser)-[:UNDERVISER]->(e:Emne)
WHERE f.etternavn = "Jenssen"
RETURN p;
```

```
+-------------------------------------------------------------------------------------------+
| p                                                                                         |
+-------------------------------------------------------------------------------------------+
| (:Person:Foreleser {etternavn: "Jenssen", ...})-[:UNDERVISER]->(:Emne {kode: "DAT107", ...}) |
+-------------------------------------------------------------------------------------------+
```

WHERE, AND & STARTS WITH

Neo4J & Cypher

- ✓ Kan lage sammenligninger og sette sammen betingelser med AND, OR og NOT som i SQL.

```
MATCH (f:Foreleser)
WHERE f.fornavn="Dag" AND f.etternavn starts with "T" RETURN f;
```

Funksjoner + sammenslåing av tekst

Neo4J & Cypher

✓ Finn antall studenter

```
MATCH (s:Student)
    RETURN count(s) AS totalt_antall_studenter;
```

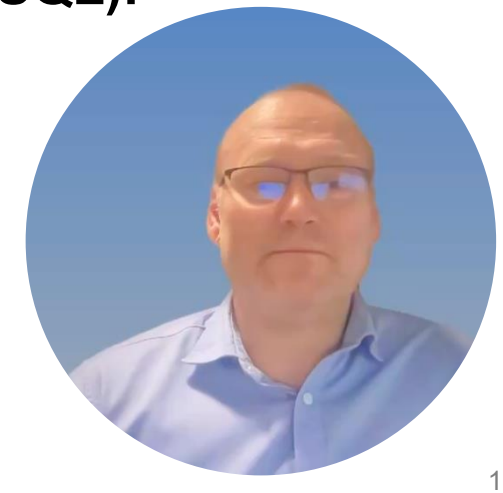
✓ Finn antall studenter som tar DAT107 emnet

```
MATCH (s:Student) -[:STUDERER]->(e:Emne {kode: "DAT107"})
    RETURN count(s) AS antall_dat107_studenter;
```

Mengdefunksjoner kan brukes uten eksplisitt gruppering (slik man må i SQL):

✓ Hvor mange emner underviser de ulike foreleserne i

```
MATCH (f:Foreleser)-[:UNDERVISER]->(e:Emne)
RETURN f.fornavn + ' ' + f.etternavn AS Foreleser,
       count(e) AS Antall_emner
ORDER BY Antall_emner DESC;
```



Avanserte spørringer

Neo4J & Cypher

✓ Klassifiser emner etter antall studenter (også de som ikke har studenter)

```
MATCH (e:Emne)
OPTIONAL MATCH (e)<-[:STUDERER]-(s:Student)
WITH e, count(s) AS ant_studenter
RETURN e.kode, e.navn, ant_studenter,
       CASE
         WHEN ant_studenter = 0 THEN "Ingen studenter"
         WHEN ant_studenter <= 20 THEN "Lite emne"
         ELSE "Stort emne"
       END AS kategori;
```

e.kode	e.navn	ant_studenter	kategori
"DAT100"	"Grunnleggende programmering"	2	"Lite emne"
"DAT107"	"Databaser"	2	"Lite emne"
"DAT201"	"Drift av Oracle databaser"	0	"Ingen studenter"

Grafalgoritmer

Neo4J & Cypher

- ✓ Funksjonen **shortestPath** beregner korteste vei mellom noder i grafen. Man kan gjøre dette for to bestemte noder, eller for alle kombinasjoner av noder.

Eksempel: *Finne (minste) antall «følgeledd» fra en bestemt person til en annen, der [:VENNER*] matcher alle mulige rettede veier langs kanter av typen [:VENNER].*

```
MATCH (p1:Person {brukernavn: 'mortene'}),
      (p2:Person {brukernavn: 'karih'})
WHERE p1.brukernavn <> p2.brukernavn
MATCH s = shortestPath((p1)-[:VENNER*]->(p2))
RETURN s;
```

```
Resultat:

| (:Person {brukernavn: "mortene", ...})-[:VENNER]->
    (:Person {brukernavn: "noran", ...}). -[:VENNER]->
      (:Person {brukernavn: "olano", ...}). -[:VENNER]->
         (:Person {brukernavn: "karih", ...}) |
```

Flytte tabelldata til Graf database

Neo4J & Cypher

- ✓ En relasjonsdatabase kan flyttes til en grafdatabase på flere måter. En mulig løsning:
  - Rader blir til noder.
  - Tabellnavn blir til nodetagger.
  - Kolonner blir til egenskaper.
  - Noen surrogatnøkler fjernes, men naturlige nøkler beholdes.
  - Fremmednøkler blir til rettede kanter.
  - Noen koblingstabeller kan erstattes av kanter med egenskaper.
  - Forholdsnavn blir til typer (på kanter).

Oppsummering

Fra objekter til NoSQL, Database History

Høgskulen på Vestlandet

NoSQL - Graf databaser

- ✓ Noder & Kanter
- ✓ Egenskaper: Nøkkel-Verdi par
- ✓ Skjemaløst
- ✓ Nettverk (i stedet for Hierarkisk struktur)

✓ **Neo4J**

- Eget **spørrespråk: Cypher**
- Distribuert, men begrenset støtte for sharding
- Støtter replikering

DEMO